

PubMatic Interview Prep

Senior Software Engineer — C/C++, Linux/Unix, Cloud

Prepared for Shreyaskumar Sharma | Generated August 13, 2026

Date & Time	Monday, August 17, 2026 · 4:00 PM IST
Mode	Zoom (link in the calendar invite)
Duration	Up to 1 hour — a single combined round
Hard requirement	Join from a laptop, not a phone. The recruiter's note says the panel may ask you to solve a problem and share your screen, and that the interview may not proceed if you join from a phone.
Arrive	5 minutes before 4:00 PM

The recruiter's prep list names five areas: programming (C/C++), DS/OOP/Algorithms/Multithreading/System Design (HLD & LLD)/Debugging, Linux OS, problem solving, and GenAI knowledge — in one hour. That is breadth, not depth. Expect the panel to move fast between areas rather than dig deep into any single one, except possibly your resume and one coding problem.

1. Read This First — The Real Challenge

Three things this panel is likely to probe, and how to handle each one honestly — without over-claiming and without underselling yourself.

1.1 The domain gap: filesystems/storage → ad serving

Your depth is in kernel-level I/O interception, encryption, and 200+ node NFS clusters. PubMatic's ad server runs at roughly 1,028B ad impressions and 2.7 trillion advertiser bids per day as of Q1 2026 — a different domain, but the underlying engineering problem rhymes: both are about squeezing latency out of a highly concurrent, distributed system under real production load. Name the gap yourself rather than hoping it goes unnoticed, then pivot to what transfers directly: concurrency control, memory management under load, systems-level debugging in production, and distributed fault tolerance.

1.2 The years-of-experience gap

The JD asks for four-plus years of C/C++ development. Your professional experience runs from roughly May 2023 to now — just over three years. Don't argue with the arithmetic if it comes up. Instead, lean on density of ownership: sole ownership of a 200+ node cluster benchmark that became a GA release gate, a sustained zero-P1 record across 2+ years, and tooling you built that got adopted org-wide. That's a lot of ground for three years, and it's a stronger answer than getting defensive about the number.

1.3 "Testing" vs. "development" framing

Your resume centers on validation, benchmarking, and debugging at scale rather than owning new product features end-to-end. If asked directly how much of your day-to-day is testing vs. writing new code, answer honestly — most of it is validation and debugging of a production-scale distributed system. Then explain why that requires

the same systems depth as building: you're diagnosing kernel-level memory corruption with GDB, designing benchmark harnesses from scratch, and increasingly building tooling (the agentic QA pipeline, the MCP integrations) rather than just running scripts. Claim exactly what you did — don't claim you built the encryption engine itself if you didn't.

*This document gives you frameworks and the facts already on your resume. For any question that asks for a specific number, metric, or story your resume doesn't spell out (e.g. exactly what broke during the 200-node test, or the specific bug in a GDB story), you know the real answer and I don't — those spots are marked **"Fill in before the interview"**. Walk in with those filled, not improvised.*

2. Your Opening Pitch — "Tell Me About Yourself"

A 60–90 second script. Practice it out loud, not just silently — the timing matters more than the exact wording.

"I'm a systems engineer with about three years of experience working at the intersection of Linux internals and distributed storage. For the last two-plus years I've been embedded at Thales, on their CipherTrust Transparent Encryption product, which does kernel-level I/O interception and encryption underneath NFS. My core work has been owning correctness and scale validation for that system — the biggest piece was designing and running a scalability evaluation of our Live Data Transformation feature across a 200-plus node NFS cluster, which was the first benchmark of its kind internally, and the thresholds I set became the actual GA release criteria for the product. Alongside that, I spend a lot of time on production debugging — memory corruption, concurrency bugs — using GDB and core dump analysis, which means I live inside the VFS layer and the kernel I/O path. I've also been building tooling on top of that: I integrated Jira and Confluence into our AI dev workflow through MCP, which got adopted org-wide, and I built an agentic QA pipeline that cuts log noise by about 94% and auto-triages failures. What draws me to this role is the scale — you're running well over a trillion ad decisions a day, and the core problems, squeezing latency out of a highly concurrent distributed system, managing memory and threads under real production load, debugging things that only break at scale, are the same problems I've been solving, just in a different domain. I'm looking to move from hardening someone else's system to building core product features where the scale is even bigger."

3. Resume Deep-Dive — Question & Answer Log

Interviewers who bother to open your resume tend to pull threads on the specific, unusual details — not the generic lines. These are the threads most likely to get pulled.

3.1 The 200+ Node NFS Cluster / LDT Scalability Evaluation

Q: Walk me through the NFS cluster scalability project. What is LDT, and what did you actually do?

A: LDT is Live Data Transformation, a CipherTrust Transparent Encryption feature that encrypts or rekeys data on a live GuardPoint without blocking application access — it does the rekey in the background instead of the traditional approach of blocking I/O during the transform. My job was to own the scalability evaluation of LDT running across a 200-plus node NFS cluster; nobody had benchmarked it at that scale internally before. I designed the test campaign: how to drive realistic concurrent I/O load across that many clients hitting the same NFS-backed GuardPoints, what to measure (throughput degradation during rekey, latency percentiles, resource consumption under LDT's own QoS scheduling), and how to catch correctness issues, not just performance ones, since a rekey has to stay crash-consistent. The thresholds that came out of it became the GA release gate for the feature.

Before the interview, fill in: the one or two concrete failure modes you actually found at scale, and the rough size of the throughput/latency thresholds you set.

Likely follow-ups:

- What broke when you scaled it up, specifically?
- How did you generate realistic concurrent load across 200+ nodes?
- How do you know your thresholds were correct and not just convenient?
- What happens if the cluster crashes mid-rekey — how does LDT recover?

3.2 Production Debugging — GDB & Core Dumps

Q: Tell me about a specific memory corruption or concurrency bug you diagnosed with GDB.

A: This is the single most important story to have ready cold, in STAR form: what the symptom was in production, how you narrowed it down (core dump inspection, backtraces, watchpoints, reproducing under load), what the actual root cause turned out to be (e.g. a race on a shared structure, a use-after-free, a buffer overrun), and what you changed to fix and prevent recurrence.

Before the interview, fill in: one real bug, start to finish, with the actual GDB commands/technique you used and the fix. Interviewers will ask for specifics ("what did the backtrace show?", "how did you confirm it was a race and not a leak?") — a vague answer here reads worse than a small bug well told.

Likely follow-ups:

- What GDB commands do you reach for first when handed a core dump with no other context?
- How do you tell a race condition apart from a memory-corruption bug from the symptoms alone?
- How would you debug a crash you can't reproduce locally?

3.3 High-Concurrency Storage Validation — Oracle / MongoDB / YCSB

Q: What did you actually validate with Oracle, MongoDB, and YCSB, and why those three?

A: These represent different I/O patterns worth stress-testing encryption against: Oracle for heavy structured/transactional I/O, MongoDB for a different concurrency and write-pattern profile, and YCSB as a configurable synthetic benchmark for tuning specific read/write mixes and concurrency levels. Running all three against CTE gave performance and stability numbers under realistic, varied production-like load ahead of each GA milestone, rather than trusting a single synthetic benchmark.

Before the interview, fill in: one specific number you're comfortable citing (e.g. throughput overhead at a given concurrency level, or a stability issue found under one of these workloads).

Likely follow-ups:

- How do you decide what concurrency level is "realistic" for a benchmark?
- What's the difference in I/O pattern between Oracle and MongoDB that matters for encryption overhead?

3.4 Zero P1 Record & Ransomware Protection Automation (+80%)

Q: You expanded ransomware protection automation coverage by 80% and eliminated a regression class. What was the automation, and what class of bug did it stop catching late?

A: CTE's ransomware protection monitors process I/O behavior for abnormal activity and blocks it before it can encrypt/corrupt guarded data. The 80% coverage expansion means significantly more of that detection logic's edge cases and regressions were caught automatically, pre-release, instead of surfacing as production escapes — which is the direct link to the zero-P1 record across 2+ years.

Before the interview, fill in: what the regression class actually was (a specific detection-logic edge case, a false positive/negative category, etc.) and roughly how the automation coverage was built out.

Likely follow-ups:

- What does "zero P1s across 2+ years" actually take, day to day — is that discipline in testing, in code review, in release process, or all three?
- Walk me through what a P1 would have looked like before this automation existed.

3.5 AI-Augmented Engineering — Jira/Confluence MCP Integration (org-wide adoption)

Q: You pioneered Jira and Confluence MCP integration, adopted org-wide. What does that actually do?

A: MCP (Model Context Protocol) lets an AI coding assistant call out to external tools directly — in this case, Jira and Confluence — instead of engineers manually copy-pasting ticket context or documentation into a chat window. Wiring that in meant the team's AI-assisted CLI and VS Code workflows could pull ticket details, search documentation, and (depending on scope) update tickets, directly from the assistant, cutting the time to go from a Jira ticket to a first-draft automation script from hours to minutes.

Likely follow-ups:

- What made this get adopted org-wide instead of staying a personal workflow tweak?
- What guardrails did you put around it, given it can act on real tickets/docs?

3.6 Agentic QA Pipeline (Claude CLI + MCP: Jira, GitLab, Confluence)

Q: Explain the agentic QA pipeline — the ~94% token reduction, the six-category failure taxonomy, and the human-in-the-loop gates.

A: The pipeline takes raw test/CI logs, which are large and noisy, and runs them through a log preprocessor that strips boilerplate and irrelevant output before anything reaches the model — that's where the roughly 94% token reduction comes from, which matters both for cost and for keeping the failure signal dense enough for the model to reason about accurately. Failures get classified into six categories (the kind of taxonomy that separates, e.g., environment/infra failures from real regressions from flaky tests), and because misclassifying a real regression as noise is costly, the pipeline has human-in-the-loop approval gates before anything is auto-closed or auto-triaged. It's now used by the engineering team for faster triage and release validation.

Before the interview, fill in: the six failure categories, if you can recall them exactly, and one concrete example of the pipeline catching (or almost missing) something important.

Likely follow-ups:

- How did you validate the log preprocessor wasn't stripping out signal along with the noise?
- What happens when the model misclassifies a failure — what's the fallback?
- Why six categories — how did you land on that taxonomy?

3.7 CI Reliability at Coriolis (flaky test elimination)

Q: How did you approach restoring CI reliability across multiple platform codebases?

A: The framework worth walking through: identifying which failures were genuinely flaky vs. real (often by re-run rate and correlation with specific environment/timing conditions), root-causing the common flaky patterns (race conditions in test setup/teardown, timing-dependent assertions, shared test state, environment dependencies), and fixing or quarantining them systematically rather than one-off. The payoff was less about a single fix and more about rebuilding engineer trust in the suite — people stop re-running failed builds "just in case" once they trust a red build means something.

Before the interview, fill in: one or two specific flaky-test root causes you actually fixed.

Likely follow-ups:

- How do you tell a genuinely flaky test from an intermittent real bug?

3.8 TaskEZ (personal project)

Q: Tell me about TaskEZ's design — why the layered architecture, why SQLite?

A: TaskEZ is a peer-reviewed task competition system with a majority-vote approval engine (PENDING → APPROVED/REJECTED). The layered architecture (models → repository → service → entry-point) keeps data access, business logic, and I/O separated, which is what makes the PostgreSQL/REST migration path realistic later without a rewrite. Data integrity is enforced at the database layer itself (FK/UNIQUE/CHECK constraints, UTC timestamps) rather than trusted purely to application logic, which matters once concurrent writers are involved.

Likely follow-ups:

- Why enforce constraints at the DB layer instead of just in application code?
- How does the majority-vote engine handle a tie?

3.9 Hackathon Win — ReadEZ (NLP)

Q: What made ReadEZ win, technically, against the other teams?

A: Frame this around scope and completeness under time pressure as a solo lead: an end-to-end pipeline covering chapter summarization, genre classification, and personalized recommendations in a single working system, not just one component done well. Be ready to name the actual NLP techniques/models you used if asked — interviewers will probe for depth here since it's an unusual credential next to a systems-heavy resume.

Before the interview, fill in: the actual models/libraries you used for summarization, classification, and recommendations.

3.10 The Title & Experience Question

Q: Your resume header says 'Staff File System Engineer,' but this JD is for a Senior role asking for 4+ years — how do you reconcile that?

A: Answer straight, don't dodge: "Staff" on the resume reflects how I've framed it for filesystem-focused roles rather than a formal title issued by Thales or Coriolis — verbally in this interview it's more accurate to describe the level as senior-track. My actual experience is about three years, running May 2023 to now, as a Coriolis contractor embedded full-time at Thales CipherTrust. I'd rather you weigh what I actually owned in that time than the label.

Likely follow-ups:

- *So what would you call your actual level, honestly?*
- *Is Coriolis your employer or is Thales?*

4. Core CS Fundamentals

Standard-depth answers for the coding/OS/multithreading portion. You know most of this cold already — use this as a fast pre-interview refresh, not first-time learning.

4.1 C/C++ & OOP

Q: What are the four pillars of OOP, and how does C++ implement each?

A: Encapsulation (private/protected members with public accessors), abstraction (abstract base classes / pure virtual functions), inheritance (public/protected/private inheritance), and polymorphism (compile-time via overloading/templates, run-time via virtual functions).

Q: How do virtual functions and vtables actually work?

A: Each class with virtual functions gets a vtable — an array of function pointers — and each object of that class carries a hidden vptr pointing to its class's vtable. A call through a base-class pointer/reference is resolved at runtime by following vptr into the vtable, which is what enables dynamic dispatch. The cost is one extra pointer per object and one indirection per virtual call.

Q: What is RAII and why does it matter?

A: Resource Acquisition Is Initialization: tie a resource's lifetime to an object's scope, acquiring in the constructor and releasing in the destructor, so the resource is freed automatically even when an exception unwinds the stack. It's directly relevant to your debugging work — a lot of the memory corruption/leak bugs you diagnose with GDB trace back to resources not following this discipline.

Q: `unique_ptr` vs `shared_ptr` vs `weak_ptr` — when do you use each?

A: `unique_ptr` for exclusive ownership with zero overhead beyond a raw pointer; `shared_ptr` when ownership is genuinely shared, at the cost of atomic refcounting overhead; `weak_ptr` to observe a `shared_ptr`-owned object without extending its lifetime, mainly to break reference cycles.

Q: What's the Rule of Three/Five, and what happens if you ignore it?

A: If a class manages a resource and needs a custom destructor, it almost certainly needs a custom copy constructor and copy assignment operator too (Rule of Three; Rule of Five adds move constructor/assignment). Ignore it and the compiler-generated versions do a shallow copy, so two objects end up owning — and eventually double-freeing — the same resource.

Q: Where does `const` correctness actually matter in a large codebase?

A: It documents and enforces intent at compile time — a `const` method promises not to mutate object state, a `const` reference parameter promises not to mutate the argument. In a codebase the size of an ad server or a distributed storage system, that compile-time contract catches an entire class of bugs before they ever reach a debugger.

Q: How do you actually find a memory leak in a C/C++ service?

A: Valgrind or AddressSanitizer for a controlled repro; for production, core dump analysis with GDB plus heap-profiling tools, correlating growth over time with specific code paths. This connects directly to your CipherTrust debugging work — be ready to describe your actual triage sequence.

Q: `malloc/free` vs `new/delete` — what's actually different?

A: `malloc/free` only allocate/release raw memory; `new/delete` additionally call the constructor/destructor, are type-safe (return the correct pointer type), and can be overloaded per-class. Mixing them (`malloc` + `delete`, or `new` + `free`) is undefined behavior.

4.2 Operating Systems & Linux Internals

This is your strongest section on paper — VFS and kernel I/O are literally your day job. Answer these with the confidence of someone who lives here, not a memorized textbook definition.

Q: Process vs. thread — what's actually different at the OS level?

A: Processes have separate address spaces, file descriptor tables, and other OS resources; threads within a process share the address space, heap, and file descriptors but get their own stack, registers, and program counter. Context switches between threads of the same process are cheaper than between processes because the page tables/TLB don't need to be flushed.

Q: Walk me through what happens on a page fault.

A: The MMU can't translate a virtual address via the page table (or the page is marked not-present), which traps into the kernel. The kernel checks if the address is valid for that process (a real fault, e.g. accessing unmapped memory, triggers a SIGSEGV) or if it's a legitimate case — demand-paged memory not yet loaded, or a swapped-out page — in which case the kernel loads the page, updates the page table, and resumes the instruction.

Q: What is the VFS layer, and why does it exist?

A: The Virtual File System is the kernel's abstraction layer that presents a single, uniform filesystem interface (open/read/write/etc.) to user space regardless of the underlying filesystem — ext4, NFS, or in your case, whatever CTE is intercepting underneath. It works through a set of common objects (superblock, inode, dentry, file) with function-pointer tables that each concrete filesystem implements. This is exactly the layer your I/O interception work sits in/near — be ready to talk about it concretely, not abstractly.

Q: What are the four necessary conditions for deadlock, and how do you break each?

A: Mutual exclusion, hold-and-wait, no preemption, and circular wait. In practice: enforce a global lock-ordering to prevent circular wait, use try-lock with backoff to avoid hold-and-wait, or use timeouts. You can't remove mutual exclusion for most real resources, so ordering is usually the practical fix.

Q: What actually happens during a system call — the user-space to kernel-space transition?

A: A user process executes a syscall instruction (e.g. syscall on x86-64), which triggers a controlled transition to kernel mode via a predefined entry point (rather than an arbitrary jump), switching the CPU's privilege level and stack. The kernel looks up the syscall number in a table, executes the handler, and returns control (and a result) to user space.

Q: Why would a high-concurrency server use epoll instead of select/poll?

A: select/poll require passing the full set of file descriptors on every call, and the kernel re-scans all of them each time — $O(n)$ per call regardless of how many are actually ready. epoll keeps the interest list in the kernel across calls and only returns the descriptors that are actually ready, which is what makes it scale to the tens of thousands of concurrent connections a high-throughput server (like an ad server handling real-time bid requests) needs.

Q: NFSv3 vs NFSv4 — what's the practical difference?

A: NFSv3 is stateless (no open/close state kept server-side, which simplifies crash recovery but complicates locking); NFSv4 is stateful, integrates locking into the protocol, supports compound operations (batching multiple ops in one RPC), and has stronger security (Kerberos) options. Given your resume's NFS depth, expect a follow-up on how CTE/LDT behaves differently depending on which one is underneath.

4.3 Multithreading & Concurrency

Q: Race condition vs. deadlock — how do you explain the difference cleanly?

A: A race condition is an outcome that depends on unpredictable timing/interleaving of concurrent operations on shared state — it may or may not corrupt data. A deadlock is a specific failure mode where two or more threads are each waiting on a resource the other holds, so none can proceed. A race can happen with no locks at all; a deadlock generally requires locks (or equivalent blocking primitives).

Q: Mutex vs. semaphore vs. spinlock?

A: A mutex enforces exclusive ownership (only the locking thread can unlock it) and typically puts waiting threads to sleep. A semaphore is a counter that allows N concurrent holders and has no ownership concept — any thread can signal it. A spinlock busy-waits instead of sleeping, which is only worth it when the expected wait is shorter than a context switch, e.g. inside kernel code or very short critical sections.

Q: Implement producer-consumer conceptually — what primitives do you need?

A: A shared bounded buffer, a mutex protecting it, and two condition variables (or one, checked twice) — one for "not full" that producers wait on, one for "not empty" that consumers wait on. Always check the predicate in a while loop, not an if, to guard against spurious wakeups.

Q: "Can multiple threads execute the same instruction?" (a question PubMatic candidates have actually been asked)

A: Yes — on a multi-core system, threads run genuinely in parallel, and there's nothing stopping two threads from being at the same program-counter address on different cores at the same instant, particularly for shared, read-only code. What actually requires synchronization isn't the instruction itself, it's shared mutable data those instructions touch — that's where cache coherence (e.g. MESI) and explicit locking come in.

Q: What is false sharing, and how would you spot it?

A: Two threads modifying logically independent variables that happen to sit on the same CPU cache line, so the cache-coherence protocol bounces that line between cores on every write, even though there's no real data dependency. You'd spot it via profiling (perf, showing unexpectedly high cache-miss/coherence traffic on otherwise-uncontended data) and fix it by padding/aligning structures so hot variables land on separate cache lines.

5. Data Structures & Algorithms

Candidate reports for PubMatic's Pune SWE interviews point to a first round mixing linked lists, sliding window/hashmap problems, and basic OS questions, with multithreaded and LLD questions in later rounds. Given you have one combined hour, expect one or two of these compressed together rather than a full separate round.

5.1 Questions specifically reported by past PubMatic candidates

- Reverse a linked list (iterative and recursive)
- **Remove a node from a singly linked list given only a pointer to that node (not the head)** — a classic trick question, see below
- Root-to-leaf path with maximum sum (root-to-max-leaf-node path)
- K-Anagrams — group or verify strings as anagrams allowing k character differences
- **Implement a circular queue using an array**, with head/tail tracked via modulo arithmetic
- Sliding window + hashmap style problems (e.g. longest substring without repeats, subarray sums)
- Process vs. thread, hashing basics, "can you make multiple threads execute the same instruction"

5.2 The two trick questions, worked

Q: Remove a node from a linked list given only a pointer to that node (no head, no previous node).

A: You can't unlink it the normal way since you have no access to the previous node's next pointer. The trick: copy the next node's value into the current node, then point `current->next` to `next->next`, and delete the (now-duplicate) next node. Effectively you're deleting the `*next*` node's memory while making the current node `*look*` like it was removed. Say the caveat out loud: this doesn't work if the given node is the tail, since there's no next node to copy from — call that out proactively, it's exactly what a strong candidate flags unprompted.

Q: Implement a circular queue backed by a fixed-size array.

A: Track head and tail indices and a count (or one wasted slot) to distinguish full from empty. On enqueue: `arr[tail] = val`; `tail = (tail + 1) % capacity`. On dequeue: `val = arr[head]`; `head = (head + 1) % capacity`. The modulo is what lets both indices wrap around the array instead of running off the end. State clearly how you're distinguishing a full queue from an empty one before you start coding — that's usually the actual thing being evaluated.

5.3 A framework for anything unseen

- State your understanding of the problem back, out loud, including edge cases (empty input, single element, duplicates) before writing anything.
- Say the brute-force approach and its complexity first, even if obviously suboptimal — it shows you can reason about complexity, and it's a safety net if you stall on the optimal one.
- Narrate your optimization: what's the bottleneck in the brute force, and what data structure removes it (hashmap for $O(1)$ lookup, two pointers to avoid a nested loop, a heap for top-K).
- State final time and space complexity without being asked.

5.4 Given four days, where to actually spend practice time

Reading answers here won't build the pattern-recognition speed a live round needs — that only comes from solving problems under time pressure. You already have a LeetCode habit (1630 rating, 350+ problems); for these four

days, bias practice toward the patterns above rather than breadth: linked-list manipulation without extra space, sliding window with a hashmap, tree root-to-leaf traversal (DFS with running sum), and one array/queue implementation problem solved from scratch without a library container.

6. System Design (HLD & LLD)

Past candidates specifically flag the LLD round as the one people under-prepare for. Given the JD lists HLD and LLD together and you only have one hour total, most likely you'll get either one focused LLD problem, or a lighter HLD conversation anchored on your NFS/CTE experience rather than a from-scratch ad-server design. Prepare both, lead with whichever the panel opens.

6.1 A framework to narrate for any design question

- **Clarify requirements:** functional scope, plus the non-functional ones that actually drive the design — latency budget, requests/sec, consistency vs. availability needs.
- **Back-of-envelope math:** rough scale numbers change the design. Worked example below.
- **High-level components:** draw/narrate the boxes before any single one goes deep.
- **Deep dive** on the 1–2 components the interviewer cares about — let them steer.
- **Bottlenecks & trade-offs:** name what breaks first at 10x scale, and how you'd address it.

6.2 Worked example: a low-latency ad request path (HLD)

If asked something like "how would you design the system that decides which ad to serve for an incoming request," here's a structure to reason through out loud (numbers below are PubMatic's own public Q1 2026 figures — citing them shows you did your homework, but treat them as illustrative, not something to defend precisely):

- **Scale check:** ~2.7T advertiser bids/day is roughly 31M/sec on average, with sharp bursts around peak traffic — design for the burst, not the average.
- **Latency budget:** real-time bidding auctions typically run on the order of ~100ms end to end, which rules out synchronous disk or slow network calls anywhere on the hot path.
- **Hot path:** edge/CDN → load balancer → ad decision service, holding targeting rules and user-profile lookups in memory or a fast cache (Redis/Memcached) rather than hitting a database per request.
- **Fan-out to bidders:** parallel async calls to downstream demand partners with a hard timeout and circuit breakers — a slow bidder must not slow down the whole auction.
- **Everything non-critical is async:** logging, billing, analytics go onto a queue (e.g. Kafka) and get processed after the response is already sent — never block the response path on them.
- **Scaling lever:** horizontal sharding by publisher/region, consistent hashing for shard routing so re-sharding doesn't reshuffle everything.

6.3 Worked example: a rate limiter (LLD)

A strong, self-contained LLD answer, and a natural fit for an ad platform (throttling requests per publisher/advertiser/IP).

- **Pick an algorithm and justify it:** token bucket allows controlled bursts and is simple to reason about; sliding-window log is precise but memory-heavy; fixed-window counter is cheap but allows edge-of-window bursts. Token bucket is the safe default to lead with.

- **Core structure:** per key (e.g. per advertiser ID), store {tokens, last_refill_timestamp, capacity, refill_rate}. On each request: compute tokens to add based on elapsed time since last_refill, cap at capacity, then allow the request only if at least one token is available and decrement it.
- **Thread safety:** guard each key's bucket with its own lock (or an atomic compare-and-swap loop) rather than one global lock, so unrelated keys don't contend.
- **Distributed version:** single-process locks don't work once you have multiple ad-server instances — move the counters into Redis and use an atomic script (Lua/EVAL) or Redis's own atomic INCR + TTL to keep the check-and-decrement atomic across instances.

7. GenAI / AI-Enabled Engineering

The JD explicitly calls for an "AI-first" engineering mindset, and PubMatic has been publicly pushing hard into this: **AgenticOS** and **Decision Fabric** (launched June 2026) run partner decisioning models directly inside the live auction path, with 20+ agents embedded across the platform and 1,000+ AI-powered deals transacted as of Q1 2026. This is a genuine area where your resume is unusually strong for the role — use it.

Q: How do you actually use AI coding tools day to day?

A: Ground this in your resume: the Jira/Confluence MCP integration (org-wide adoption) and the agentic QA pipeline (Claude CLI, ~94% token reduction via log preprocessing, six-category failure taxonomy, human-in-the-loop gates). The through-line: you use these tools to remove grunt work (context-gathering, log triage) while keeping a human decision point wherever the cost of a wrong automated call is high.

Q: What's a time an AI tool gave you a wrong or risky suggestion, and how did you catch it?

A: You need a real one here — interviewers ask this specifically to filter out people who use AI tools uncritically. A generic answer ("I always review the code") reads as evasive.

Before the interview, fill in: one specific instance: a hallucinated API, a subtly wrong concurrency pattern, or a suggestion that would've broken under your actual load, and how your test/review process caught it.

Q: What's your view on where LLM-generated code falls short in systems-level work?

A: A defensible answer: LLMs are strong at boilerplate, test scaffolding, and pattern-matching against common code, but weak at reasoning about the specific concurrency/memory-ordering guarantees of your actual system, and can confidently suggest APIs or behaviors that don't exist or don't apply to your kernel version/environment. That's exactly why your QA pipeline keeps a human-in-the-loop gate rather than auto-approving.

8. Behavioral & Culture Fit

Q: Why PubMatic?

A: Anchor on something specific, not generic: PubMatic is running real decisioning models (AgentiOS / Decision Fabric) directly inside the live auction path at trillion-plus daily scale — that's the kind of production AI-systems problem, at a scale and latency budget, that you don't get many places to work on. Pair that with the JD's explicit ask for AI-first engineers, which plays directly to what you've already built (MCP integrations, the agentic QA pipeline).

Q: Why leave your current role?

A: Keep this forward-looking and specific to what you want, not a complaint about Thales/Coriolis: you've spent two-plus years owning validation, benchmarking, and debugging for someone else's product at scale, and you're looking to move into building core product features in a similarly high-scale environment.

Before the interview, fill in: phrase this in your own words — keep it about what you're moving toward, not what you're moving away from.

Q: Tell me about a conflict or disagreement with a teammate or lead.

A: Standard STAR shape: the situation, the specific disagreement (ideally a technical one you can speak to credibly, e.g. a disagreement over a benchmark methodology or a fix approach), how you resolved it (data, escalation, compromise), and the outcome.

Before the interview, fill in: one real instance from Thales or Coriolis.

Q: Tell me about a failure.

A: Pick something real and moderate in size — not a catastrophe, not something so minor it shows no risk was taken. What you'd want to demonstrate: you caught it, you owned it without deflecting, and you changed something concrete afterward (a process, a test, a review habit) so it couldn't recur.

Before the interview, fill in: one real instance; avoid picking something that overlaps with the zero-P1 claim on your resume in a way that undercuts it.

Q: What's your biggest strength, and your biggest weakness, for this role?

A: Strength: production debugging depth at scale — you can point to the zero-P1 record and the 200-node benchmark as evidence, not just a claim. Weakness: pick something real and specifically relevant to the transition you're making — e.g. your hands-on product-feature-development experience (as opposed to validation/hardening) is lighter than your systems-debugging experience, and you're actively addressing that by targeting exactly this kind of role.

Q: Where do you see yourself in a few years?

A: Tie it to growing from validating/hardening systems into owning core feature development at increasing scope — consistent with why you framed the "why leave" answer the way you did, so the two answers reinforce each other instead of contradicting.

9. Smart Questions to Ask the Panel

Ending with generic questions ("what's the culture like?") wastes the one signal you get to send about how seriously you researched the role. These are specific to what's public about PubMatic right now.

- "Now that Decision Fabric is running partner decisioning models directly inside the live auction path, how has that changed the reliability bar for the core ad-serving team specifically?"
- "For a Senior Engineer on this team, what's the actual split between building new ad-serving features versus hardening and scaling the existing pipeline?"
- "What does production ownership/on-call look like for this team?"
- "Is the Pune engineering org fully owning a service end-to-end, or is this more of a shared component with teams elsewhere?"
- "What would success look like for someone in this role at the six-month mark?"
- "What's the hardest technical problem the team is actively wrestling with at your current scale?"

10. Day-Of Checklist

Before you join

- Laptop, not phone — confirmed hard requirement from the recruiter.
- Stable connection; close bandwidth-heavy apps/tabs.
- Have an editor and a compiler ready to share instantly: a blank VS Code window plus a terminal with g++ and python3 confirmed working, in case you're asked to code live.
- Join the Zoom link 5 minutes early.
- Resume and this document open on a second screen or phone for a glance beforehand — not during the interview itself.

Final review order, in priority

- 1. Your 90-second opening pitch (Section 2) — out loud, twice.
- 2. The four "fill in before the interview" gaps in Section 3 — these are your real, specific war stories; don't walk in with placeholders still unfilled.
- 3. Section 4.2 (OS/VFS) — your strongest ground, make sure it's fluent, not just familiar.
- 4. The two trick DSA questions in Section 5.2 — these are specifically reported repeats.
- 5. The rate limiter LLD sketch in Section 6.3.